

Defining Your Boundaries: Where to Stop

To complete this capstone within your **35–50 hour time budget**, you must strictly resist scope creep. This project is a backend validation of an AI safety system, not a consumer application.

Use this scope checklist to keep your implementation lean and focused on passing the review:

 What to Build (Keep It Lean)	 What to Skip (Scope Creep)
~50 Image Assets: A small folder containing 10 images each for 5 categories (fox, wolf, dog, bear, deer).	Massive Image Libraries: Thousands of unorganized files or broad scrapes that complicate debugging.
API Endpoints or Terminal Table: Simple JSON payloads for your <code>GET</code> and <code>POST</code> calls, or a basic terminal UI printout.	A Web Frontend UI: No React, Vue, HTML pages, or custom dashboard layouts are needed.
Single Vision + Embedding Model: Stick strictly to Gemini Flash and a single text vector model (like <code>all-MiniLM-L6-v2</code>).	Model Comparison Engines: Don't waste time benchmarking different AI versions against each other.
Lightweight Local Cache/DB: A simple local database setup (like SQLite or an in-memory JSON registry) to store embeddings.	Production Vector Databases: Skip setting up complex external cloud clusters like Pinecone or Milvus.

Seed Data & Labeled Validation Matrix

This **15-item evaluation set** satisfies your requirement to expand slightly on the Challenge 2 seed data. You can copy this matrix directly into your automated tests (`test_suite.py`) to verify your system's performance and establish your precision metrics.

```
python
# test_suite.py
EVALUATION_SEED_DATA = [
```

```
# --- FOX CATEGORY MATCHES & TRAPS ---
{
  "post_id": "p_01",
  "post_text": "The stealthy hunting techniques of solitary red fox
  "image_desc": "A bright red fox stepping softly through thick aut
  "expected": "MATCH",
  "notes": "Direct concept match."
},
{
  "post_id": "p_02",
  "post_text": "Understanding the lifecycle of the wild vixen and h
  "image_desc": "A mother red fox (Vulpes vulpes) nursing her small
  "expected": "MATCH",
  "notes": "Synonym match (vixen/Vulpes vulpes -> red fox)."
},
{
  "post_id": "p_03",
  "post_text": "An in-depth look at how red foxes adapt to freezing
  "image_desc": "A grey timber wolf pacing across a frozen mountain
  "expected": "REJECT",
  "notes": "The Wolf Trap: Mismatch Guard must catch this close can
},
{
  "post_id": "p_04",
  "post_text": "Urban sightings of small red-furred wild canines ra
  "image_desc": "A golden retriever dog sitting calmly beside a met
  "expected": "REJECT",
  "notes": "The Dog Trap: Generic domestic canine overlap."
},

# --- WOLF CATEGORY MATCHES & TRAPS ---
{
  "post_id": "p_05",
  "post_text": "The intricate pack hierarchy and communication of w
  "image_desc": "An alpha grey wolf howling on a rocky overlook sur
  "expected": "MATCH",
  "notes": "Direct category match."
},
{
  "post_id": "p_06",
  "post_text": "Tracking the movements of Canis lupus packs across
  "image_desc": "A lone timber wolf stepping out from a dense snow-
  "expected": "MATCH",
  "notes": "Scientific name synonym match."
```

```

    },
    {
      "post_id": "p_07",
      "post_text": "Apex predators of the northern woodlands: The timbe
      "image_desc": "A red fox curled up asleep in the hollow of an old
      "expected": "REJECT",
      "notes": "Reverse Trap: Mismatch Guard must catch this fox/wolf m
    },

# --- BEAR CATEGORY MATCHES & TRAPS ---
{
  "post_id": "p_08",
  "post_text": "Grizzly bears fattening up on river salmon before t
  "image_desc": "A massive brown grizzly bear catching a leaping sa
  "expected": "MATCH",
  "notes": "Direct concept match."
},
{
  "post_id": "p_09",
  "post_text": "Hibernation patterns of Ursus arctos in North Ameri
  "image_desc": "A large black bear walking along a fallen log in a
  "expected": "MATCH",
  "notes": "Genus match for brown/black bear variants."
},
{
  "post_id": "p_10",
  "post_text": "Safety guidelines when hiking through deep grizzly
  "image_desc": "A majestic bull elk standing in a misty clearing w
  "expected": "REJECT",
  "notes": "Complete species mismatch."
},

# --- DEER/DEEPER TRAPS ---
{
  "post_id": "p_11",
  "post_text": "Whitetail deer bounding gracefully through open gra
  "image_desc": "A whitetail doe with two spotted fawns grazing in
  "expected": "MATCH",
  "notes": "Direct match."
},
{
  "post_id": "p_12",
  "post_text": "The velvet shedding cycle of male cervid antlers in
  "image_desc": "A large male mule deer rubbing its fuzzy antlers a

```

```

    "expected": "MATCH",
    "notes": "Advanced vocabulary match (cervid -> deer family).",
  },
  {
    "post_id": "p_13",
    "post_text": "Foraging habits of forest herbivores like deer and
    "image_desc": "A brown grizzly bear scratch-marking its back agai
    "expected": "REJECT",
    "notes": "Herbivore vs. Omnivore/Predator mismatch."
  },

# --- NO CONFIDENT MATCH TRAPS ---
{
  "post_id": "p_14",
  "post_text": "A technical breakdown of mechanical engine performa
  "image_desc": "A group of whitetail deer drinking from a shallow
  "expected": "REJECT",
  "notes": "Out of domain text: System must say 'No confident match
},
{
  "post_id": "p_15",
  "post_text": "Deep sea diving exploration near colorful coral ree
  "image_desc": "A lone grey wolf running down a gravel path.",
  "expected": "REJECT",
  "notes": "Out of domain text: System must say 'No confident match
}
]

```

Use code with caution.

Formatting Your Baseline Build Log

To document this setup phase in your workspace files, add this entry to your repository tracking log:

markdown

```

### Phase 3 Definition of Scope Bounds & Validation Seed

```

```

- **What I Did:** Restricted project scope to a 50-image corpus across 5

```

- **AI Tool Assistance:** Helped expand the 15-item test matrix with clear
- **Human Course Corrections:** Stripped out a planned integration with a

Use code with caution.

If you are ready to implement this test suite into your workflow, let me know:

- Would you like me to build a **mock data loading utility** that feeds this exact validation seed matrix directly into your `FlyRankImageMatcher` class to check your error margins?
- Or should we write the **database interaction models** to save these results to your local repository cache?

